Name – Nishchay Agrawal

Roll No – 202501100700098

Branch – Electronics and Communications

Section - B

Problem Statement: Smart Payment Processing System using OOP in Python

Design and implement a **Smart Payment Processing System** using Object-Oriented Programming concepts in Python. The system should allow a user to make payments using different payment methods such as **Credit Card, UPI, PayPal, and Digital Wallet**. Each payment method must follow a **common interface** but implement **different business logic** for computing the final payable amount.

Create an abstract base class named **Payment** that stores the user name and declares an abstract method `pay(amount)` which must be implemented by all child classes. The base class should also contain a concrete method `generate_receipt()` that prints the user name, original amount, and final amount paid after processing.

Implement the following payment types by inheriting from the Payment class:

- **CreditCardPayment**: Apply a **2% gateway fee** on the transaction amount and then apply **18% GST on the gateway fee**. The final amount should be computed by adding the original amount, gateway fee, and GST.
- **UPIPayment**: Provide a **cashback of ₹50** if the payment amount is greater than 1000. Otherwise, no cashback is applied. The final payable amount should be calculated after subtracting cashback.
- **PayPalPayment**: Apply a **3% international transaction fee** and an additional **fixed currency conversion fee of ₹20**. These charges should be added to the original amount to compute the final payment.
- **WalletPayment**: This payment method should maintain a wallet balance. If the payment amount exceeds the available balance, the transaction should fail. Otherwise, deduct the amount from the wallet and update the remaining balance.

Create a function `process_payment(payment, amount)` that accepts any payment object and calls its `pay()` method. This function should demonstrate **runtime polymorphism**, where the same function behaves differently depending on the payment type object passed.

Finally, create objects of all payment classes and process multiple transactions to demonstrate **abstraction, inheritance, and polymorphism** working together in a real-world payment processing system.

Submission Link: https://docs.google.com/forms/d/e/1FAIpQLSdRzOdWF6qk-puLAjvc-1P3rs4K037EmdPsxK0YTi1b8JNwwQ/viewform?usp=sharing&ouid=117626677736717658229

casestudy3.py

```python
from abc import ABC, abstractmethod

# Abstract Base Class
class Payment(ABC):
    def __init__(self, user_name):
        self.user_name = user_name
        self.original_amount = 0
        self.final_amount = 0

    @abstractmethod
    def pay(self, amount):
        pass

    def generate_receipt(self):
        print("\n----- PAYMENT RECEIPT -----")
        print(f"User Name       : {self.user_name}")
        print(f"Original Amount : ₹{self.original_amount}")
        print(f"Final Amount    : ₹{self.final_amount}")
        print("----------------------------\n")


# Credit Card Payment
class CreditCardPayment(Payment):
    def pay(self, amount):
        self.original_amount = amount

        gateway_fee = amount * 0.02
        gst = gateway_fee * 0.18

        self.final_amount = amount + gateway_fee + gst

        print("\nProcessing Credit Card Payment...")
        self.generate_receipt()


# UPI Payment
class UPIPayment(Payment):
    def pay(self, amount):
        self.original_amount = amount

        cashback = 50 if amount > 1000 else 0
        self.final_amount = amount - cashback

        print("\nProcessing UPI Payment...")
        if cashback:
            print("Cashback Applied: ₹50")
        else:
            print("No Cashback Applied")

        self.generate_receipt()

```

```python

# PayPal Payment
class PayPalPayment(Payment):
    def pay(self, amount):
        self.original_amount = amount

        international_fee = amount * 0.03
        conversion_fee = 20

        self.final_amount = amount + international_fee + conversion_fee

        print("\nProcessing PayPal Payment...")
        self.generate_receipt()


# Wallet Payment
class WalletPayment(Payment):
    def __init__(self, user_name, balance):
        super().__init__(user_name)
        self.balance = balance

    def pay(self, amount):
        self.original_amount = amount

        print("\nProcessing Wallet Payment...")

        if amount > self.balance:
            print("Transaction Failed: Insufficient Balance!")
            return

        self.balance -= amount
        self.final_amount = amount

        print(f"Remaining Wallet Balance: ₹{self.balance}")
        self.generate_receipt()


# Polymorphic Function
def process_payment(payment, amount):
    payment.pay(amount)


# ------------------ MAIN PROGRAM ------------------

user_name = input("Enter your name: ")
wallet_balance = float(input("Enter initial wallet balance: "))

wallet = WalletPayment(user_name, wallet_balance)

while True:
    print("\n===== PAYMENT MENU =====")
    print("1. Credit Card")
    print("2. UPI")
    print("3. PayPal")
```

```python
    print("4. Wallet")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 5:
        print("Thank you for using the system!")
        break

    amount = float(input("Enter amount to pay: "))

    if choice == 1:
        payment = CreditCardPayment(user_name)
    elif choice == 2:
        payment = UPIPayment(user_name)
    elif choice == 3:
        payment = PayPalPayment(user_name)
    elif choice == 4:
        payment = wallet
    else:
        print("Invalid choice! Try again.")
        continue

    process_payment(payment, amount)
```